MimicPlay

通过观察人类操作学习长时序模仿

核心方法、数学原理与 GMM 实现

作者：Yang kunpeng

论文链接：arXiv:2302.12422

目录

第一章　MimicPlay 核心方法与训练推理流程　1

第二章　MimicPlay 核心数学原理　3

第 1 章

MimicPlay 核心方法与训练推理流程

MimicPlay 面向长时序机器人操作，将规划与控制分开学习：先利用人类自由操作视频学习高层计划，再利用少量机器人遥操作示范学习低层控制。高层规划任务过程中接下来应如何移动、与何处交互；低层结合机器人视觉与本体状态，将计划转化为可执行的控制动作。整个方法包含两个训练阶段和一个测试阶段。

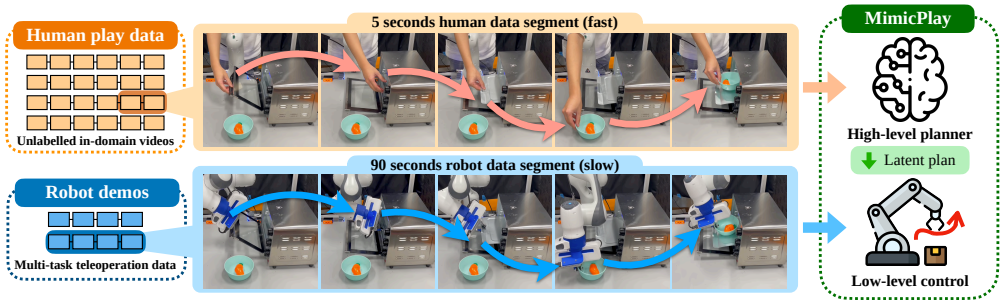


图 1.1: 人类完成操作任务通常比遥操作机器人更快，因此可以利用人类 Play 数据学习高层规划，减少机器人示范数据需求。

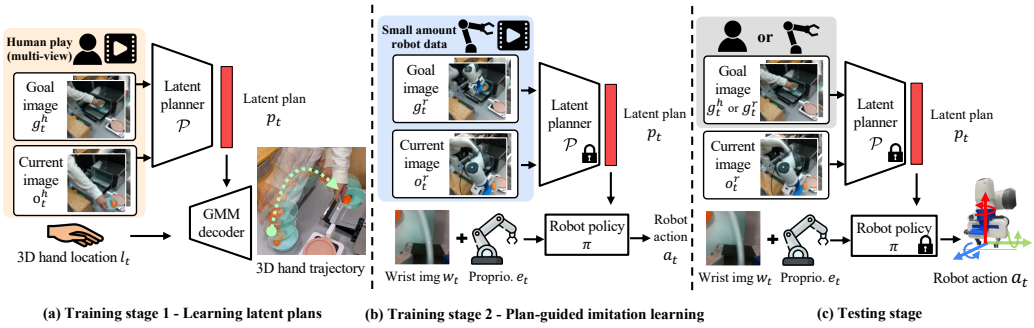


图 1.2: 方法总览。(a) 使用人类 Play 数据训练高层轨迹生成模型；(b) 使用机器人遥操作数据训练由预训练 Planner 引导的低层控制器；(c) 测试时以任务视频为提示，由高层计划引导机器人完成任务。

第一阶段：使用人类 Play 数据训练高层 Planner。人类操作者在目标环境中自由地与物体交互，不必像机器人遥操作示范那样逐条完成指定任务。每个环境收

集约 10 分钟的人类 Play 视频。为恢复三维运动，使用两台经过标定的相机，从两个视角检测手部位置并重建三维手部轨迹。高层模型以当前人类视频帧 o_t^h、目标图像 g_t^h 及当前位置 l_t 为条件，从当前观察与目标图像的视觉特征中编码出潜在计划 p_t，再根据 p_t 和 l_t 预测未来三维手部轨迹的概率分布。监督目标是从视频中提取的真实未来手部轨迹 τ_t，而不是潜在特征 p_t 本身。

人类视频与机器人相机图像存在视觉域差异。为使高层计划能够迁移到机器人观察，训练时对齐两种域的视觉特征分布，并加入 KL 散度损失。该对齐不要求人类视频帧与机器人视频帧逐帧配对。高层 Planner 训练完成后，其潜在计划用于指导低层策略。

第二阶段：使用机器人遥操作示范训练低层 Policy。低层训练使用人类操作者通过机器人遥操作系统采集的机器人示范。每项任务通常收集 20 条示范，Kitchen 环境的部分实验使用 40 条。训练时，从机器人示范序列中选择未来目标图像：目标帧设为当前帧之后 H 个时间步的图像，其中 H 在 $[200, 600]$ 范围内均匀采样，对应约 10–30 秒的未来跨度。

训练低层策略时，高层 Planner 已预训练并被冻结。Planner 根据机器人当前观察与目标图像产生潜在计划 p_t。低层策略将潜在计划、腕部相机观察 w_t 与机器人本体状态 e_t 组成时间序列输入 Transformer。Transformer 利用历史信息得到动作预测特征，再由动作预测头输出动作概率分布。真实机器人动作 a_t^* 用于计算负对数似然损失；反向传播更新低层策略，而不更新被冻结的高层 Planner。这样的分工让低层策略专注于学习机器人本体控制，而不必独立承担长时序任务规划。

第三阶段：测试与机器人执行。测试时输入一段任务视频作为提示，视频可以来自人类操作，也可以来自机器人遥操作。视频被转换为图像序列；每个控制时刻，系统选择目标图像，结合机器人当前观察调用高层 Planner，再由低层策略产生当前动作。机器人执行动作后，系统读取新的观察，并更新参考视频中的目标位置。目标帧索引由参考视频的当前位置与目标间隔共同确定，并根据机器人末端位置与参考轨迹窗口中最近位置进行更新，而不是每一步固定增加一帧。整个过程是逐步生成、逐步执行的闭环控制，而不是一次性生成完整动作序列。

真实机器人实验与仿真设置的数据来源不同。真实机器人实验使用人类 Play 数据训练高层 Planner，再用机器人遥操作数据训练低层 Policy；仿真环境没有对应的人类 Play 数据，因此使用机器人 Play 数据训练高层与低层。当前仿真配置中，高层未来轨迹向量为 30 维，对应 10 个三维位置；低层动作向量为 7 维，动作 GMM 使用 5 个分量。

第 2 章

MimicPlay 核心数学原理

先区分三个对象。潜在计划 p_t 是神经网络产生的低维特征向量；未来轨迹 τ_t 是一串未来三维位置组成的向量；机器人动作 a_t 是低层控制器在当前控制时刻输出的动作向量。p_t 本身既不是轨迹，也不是概率密度。高层模型根据 p_t 与当前位置对未来轨迹建立概率分布；低层模型则根据 p_t、机器人观察和历史信息对动作建立概率分布。

高层 Planner 的潜在计划与轨迹建模。视觉编码器 E 处理当前图像与目标图像，再由 MLP 编码为潜在计划：

$$p_t = F_\phi\big(E(o_t^h), E(g_t^h)\big).$$

轨迹解码器再结合 p_t 与当前位置 l_t，输出 GMM 参数。高层 GMM 建模的是未来轨迹 τ_t，不是潜在向量 p_t：

$$p_\theta(\tau_t \mid p_t, l_t) = \sum_{k=1}^{K} \eta_{t,k}\, \mathcal{N}(\tau_t \mid \mu_{t,k}, \Sigma_{t,k})\,.$$

其中，K 是高斯分量数，$\eta_{t,k}$ 是第 k 个分量的混合权重，$\mu_{t,k}$ 是该分量的轨迹均值，$\Sigma_{t,k}$ 是协方差矩阵，并且 $\eta_{t,k} \geq 0$、$\sum_k \eta_{t,k} = 1$。多个高斯分量可以表示同一目标对应的多种合理运动策略，避免将多种可能的运动简单平均成一条轨迹。当前仿真配置使用 $K = 5$ 个分量，高层轨迹向量维度为 30。

高层模型通过真实轨迹的负对数似然训练：

$$\mathcal{L}_{\text{GMM}} = -\mathbb{E}_{\tau_t^*}\left[\log \sum_{k=1}^{K} \eta_{t,k} \mathcal{N}(\tau_t^* \mid \mu_{t,k}, \Sigma_{t,k})\right].$$

τ_t^* 是从训练数据中提取的真实未来轨迹。网络先根据观察与目标图像预测 GMM 参数，再将真实轨迹代入预测分布计算对数概率密度。真实轨迹参与损失计算，但不作为生成 GMM 参数的输入。由于这里是连续向量的概率密度，密度值本身可以大于 1；对整个轨迹空间积分为 1 的是完整概率密度函数，而不是每个位置的函数值。

为对齐视觉域，$\mathcal{Q}^h = E(o^h)$ 与 $\mathcal{Q}^r = E(o^r)$ 分别表示人类域和机器人域的视觉特征分布。每个 batch 中统计两类特征的均值与方差，并用

$$\mathcal{L}_{\mathrm{KL}} = D_{\mathrm{KL}}(\mathcal{Q}^r \| \mathcal{Q}^h)$$

缩小两域特征分布的差异。高层总目标写为

$$\mathcal{L} = \mathcal{L}_{\mathrm{GMM}} + \lambda \mathcal{L}_{\mathrm{KL}},$$

其中 λ 控制域对齐损失的权重。采用跨域训练时，高层目标包含轨迹负对数似然与 KL 散度两项。仅使用机器人 Play 数据的仿真训练路径中，`Highlevel_GMM_pretrain` 只计算轨迹负对数似然，没有显式计算 KL 项。

低层 Policy 的动作建模。低层策略将腕部相机特征 w_t、本体状态 e_t 和潜在计划 p_t 组成一个时刻的输入：

$$s_t = [w_t, e_t, p_t].$$

Transformer 对历史输入序列进行建模，得到动作预测特征 x_t；MLP 动作预测头再输出 GMM 的均值、标准差参数与混合权重。低层策略所建模的是当前动作的条件分布：

$$\pi_\psi(a_t \mid s_{1:t}) = \sum_{k=1}^{K} \eta_{t,k}\, \mathcal{N}(a_t \mid \mu_{t,k}, \Sigma_{t,k})\,.$$

当前低层配置使用 5 个高斯分量，每个动作向量为 7 维。给定某个分量后，各动作维度按对角高斯方式建模，亦即协方差矩阵为对角矩阵，其对角元素是标准差的平方。

低层策略用真实机器人动作训练，损失为

$$\mathcal{L}_{\mathrm{action}} = -\mathbb{E}_{(s_{1:t}, a_t^*)} \left[\log \pi_\psi(a_t^* \mid s_{1:t})\right].$$

该目标鼓励模型为示范中的真实动作分配较高的概率密度。梯度更新低层 Transformer 和动作 GMM 参数预测头；第二阶段通过 `torch.no_grad()` 获取冻结 Planner 的潜在特征，不将低层损失反向传播到高层 Planner。

GMM 的实际构建。在 PyTorch 实现中，`GPT.py` 通过 `import torch.distributions as D` 将 `torch.distributions` 简写为 `D`。低层网络先产生 `x_means`、`x_scales` 和 `x_logits`。`x_means` 是分量均值参数；`x_scales` 经 Softplus 与最小标准差处理后成为标准差，不是方差；`x_logits` 是混合权重的未归一化分数。相关实现如下：

```
x_means = torch.tanh(x_means)
x_scales = self.activations[self.std_activation](x_scales) + self.min_std

component_distribution = D.Normal(
    loc=x_means, scale=x_scales
)
component_distribution = D.Independent(
    component_distribution, 1
)
mixture_distribution = D.Categorical(
    logits=x_logits
)
dists = D.MixtureSameFamily(
    mixture_distribution=mixture_distribution,
    component_distribution=component_distribution
)
```

`D.Normal` 根据均值与标准差建立各维正态分布；`D.Independent(..., 1)` 将最后一个动作维度整体视为一个事件，因此一个高斯分量对应完整的 7 维动作向量，而不是 7 个互不相关的输出事件；`D.Categorical(logits=x_logits)` 将分量分数转换为混合权重；`D.MixtureSameFamily` 把分类分布与高斯分量组合为完整 GMM。`torch.tanh` 限制分量均值的范围，Softplus 确保标准差为正，`min_std` 防止标准差过小。低层 `GPT_wrapper` 使用上述方式构建动作 GMM；高层 `GMMActorNetwork` 采用相同的分布组合方式，但建模对象是未来轨迹，而不是机器人动作。

训练时，将真实轨迹代入预测分布计算对数概率密度。高层轨迹损失的核心计算为：

```
log_probs = dists.log_prob(
    batch["obs"]["robot0_eef_pos_future_traj"]
)
action_loss = -log_probs.mean()
```

真实机器人学习设置中，高层监督对象是从人类视频中检测并重建的三维手部未来轨迹；仿真设置中的标签字段 `robot0_eef_pos_future_traj` 对应机器人末端的未来位置。低层使用 `dists.log_prob(batch["actions"])` 计算示范动作的对数

密度，再取负并求平均。log_prob() 返回对数概率密度，而不是要求落在 0 到 1 之间的概率值。

从 GMM 采样得到动作。低层推理时，网络先预测当前状态下的 GMM，再执行 return dists.sample()。从数学上看，采样包含两个步骤：

$$z_t \sim \text{Categorical}(\eta_{t,1}, \ldots, \eta_{t,K}),$$

先根据混合权重抽取一个高斯分量编号；再从该分量的多维高斯分布中采样：

$$a_{t,j} = \mu_{t,z_t,j} + \sigma_{t,z_t,j}\epsilon_j, \qquad \epsilon_j \sim \mathcal{N}(0,1).$$

把各维采样值组合起来，得到完整动作向量 a_t。公开仿真配置中，该向量为 7 维。因为 GMM 从一开始定义的随机变量就是动作，所以采样结果本身就是动作，不需要再经过一个神经网络解码器。执行后，环境返回新的观察，策略在下一控制时刻重新预测并采样动作。

高层轨迹 GMM 与低层动作 GMM 的推理方式不同。高层分布在数学上也可以采样得到一条多维轨迹，但在 _get_latent_plan() 中，使用 dists.mean 取得高层分布的均值，而不是调用 dists.sample()。GMM 的均值为

$$\mathbb{E}[\tau_t] = \sum_{k=1}^{K} \eta_{t,k}\mu_{t,k}.$$

代码同时返回 MLP 产生的潜在特征 mlp_out；轨迹分布的均值 act_out 与潜在计划特征不是同一个对象。在 get_action() 中，轨迹分布均值与潜在特征分别写入 guidance 和 latent_plan。GPT_wrapper.forward_step() 构造低层输入时读取的是 latent_plan、腕部图像特征和机器人位姿特征，并不读取 guidance。因此，在这一实现中，高层轨迹均值并未直接作为低层网络输入；低层实际使用的是潜在计划及机器人当前观察和位姿特征。